



Orientação a Objetos

Atributos estáticos, Herança e Encapsulamento

Relacionamento entre classes



**Criar um sistema orientado
a objetos tem a ver com
descobrir suas peças e
como elas se encaixam!**

Atributos estáticos

Algumas classes possuem variáveis que são compartilhadas com todas as instâncias.



Classe

Instâncias possuem valores que são diferentes para cada uma delas.



Instancias

Atributos estáticos



a variável precisa
ter o modificador
static

```
class Gato {  
    static populacao: number = 0;  
    constructor(public nome: string) {  
        // 'Gato.populacao' se refere à CLASSE  
        Gato.populacao++;  
    }  
}
```

Atributos estáticos

Não precisa de uma
instância para acessar um
membro estático...



Classe e Objeto

Já entendemos como as classes são as abstrações a partir das quais são criados os objetos!



Abstrato



Concreto

Níveis de abstração

 **Felino**
Mais genérico



Mais específico
Gato de Rua 



**Todos são
abstrações!**

SOFTEx
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Abstrações parecidas

```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
}
```



```
class Gerente{  
    nome?: string  
    idade?: number  
    salario?: number  
    bonus?: number  
}
```



Duplicação de código

```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
}
```



Duplicado!

```
class Gerente{  
    nome?: string  
    idade?: number  
    salario?: number  
    bonus?: number  
}
```



Como evitar esse código duplicado?



SOFTEx
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO

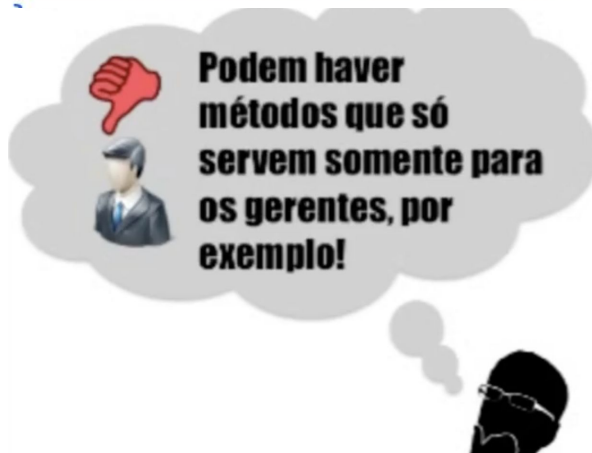
```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
    bonus?: number  
}
```



**E se tiver uma classe com
todos os atributos? O
empregado pode receber
zero no bônus!**



```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
    bonus?: number  
  
    contratar(nome: string){}  
    demitir(e: Empregado){}
```



```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
    bonus?: number  
    periodoSupervisao?: string  
    sessaoSupervisao?: string  
}
```



MAS



é um



SOFTEx
PERNAMBUCO

 **Softex**

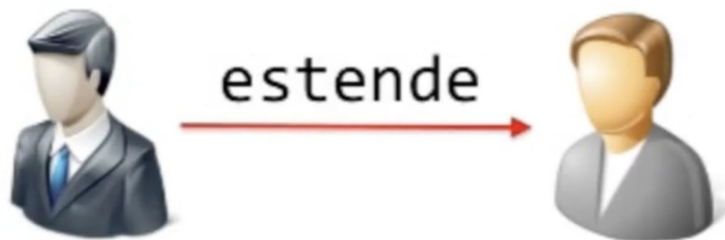
MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO

**A herança é um recurso
da orientação a objetos
que permite que
classes especializem
ou generalizem outras
classes.**



Herança



A partir da herança é possível estender uma classe, adicionando atributos e modificando comportamento.

Herança



SUPERCLASSE



estende



SUBCLASSE

SOFTEx
PERNAMBUCO

 **Softex**

MINISTERIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Herança

Question

Que outras classes poderiam ser subclasses de Funcionario?

☒ Diretor

✔ Correct

☐ Cliente

☒ Supervisor

✔ Correct

☐ Pessoa

Continue

Skip



Herança

```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
}
```



*herda métodos e
atributos*

```
class Gerente extends Empregado{  
    bonus?: number  
}
```



Herança



PODE



```
class Gerente extends Empregado{  
    bonus?: number  
}
```



Adicionar métodos



Adicionar atributos



Modificar métodos



Herança



Remover métodos



Remover atributos



Estender outra classe

```
class Gerente extends Empregado{  
    bonus?: number  
}
```



NÃO PODE



Herança



**Ao estender uma classe é
preciso honrar o contrato
da superclasse, a partir
de seus métodos e
atributos.**

Question

É possível remover, na classe, um método da superclasse?

☐ Sim

☒ Não

✔ Correct



Herança

A herança múltipla existe em algumas linguagens



Sobrescrita de Métodos

```
class Empregado{  
    liquido(salario: number){  
        return salario * 0.85  
    }  
}
```

precisa modificar o comportamento

```
class Gerente extends Empregado{  
    bonus?: number  
    //liquido() precisa adicionar o bonus  
}
```



Sobrescrita de Métodos

```
class Gerente extends Empregado{  
    bonus: number = 0  
    liquido(salario: number){  
        return salario * 0.85 + this.bonus  
    }  
}
```

**Uma classe pode
sobrescrever
métodos da
superclasse.**



Herança



**O contrato é
o mesmo!**



**O comportamento
é diferente!**

Herança

```
class Empregado{  
    liquido(salario: number){  
        return salario * 0.85  
    }  
}
```

*Se mudarem as
taxas preciso
modificar em dois
locais!*

```
class Gerente extends Empregado{  
    bonus: number = 0  
    liquido(salario: number){  
        return salario * 0.85 + this.bonus  
    }  
}
```



Herança

```
class Gerente extends Empregado{  
    bonus: number = 0  
    liquido(salario: number){  
        return super.liquido(salario) + this.bonus  
    }  
}
```



Você pode usar
super para invocar
o método na
superclasse!

Exercício 1

Exercício de Herança e Sobrescrita em TypeScript

O objetivo é modelar uma hiarquia de contas bancárias simples, utilizando herança para reutilizar o comportamento básico de uma conta e sobrescrever o método de saque na conta especializada.

Etapa	Classe	Atributos (Campos)	Métodos	Lógica Esperada
1	ContaCorrente (Super Classe)	saldo: number	depositar(valor: number) sacar(valor: number)	Adiciona <code>valor</code> ao <code>saldo</code> . Lógica Padrão: Só permite o saque se o <code>saldo</code> for maior ou igual ao <code>valor</code>
2	ContaEspecial (Sub Classe)	limite: number	sacar(valor: number)	Lógica Sobrescrita: Permite o saque se o <code>saldo</code> mais o <code>limite</code> for maior ou igual ao <code>valor</code>



Exercício 1

Exemplo de Teste (Saída Esperada):

Para a classe `ContaEspecial` com `saldo = 500` e `limite = 1000`:

Ação	Resultado Esperado
<code>conta.sacar(800)</code>	Sucesso - Novo saldo: —300
<code>conta.sacar(800)</code>	Falha - Saldo insuficiente
<code>conta.depositar(100)</code>	Sucesso (Novo saldo: —200)

